Cpr E 4890: Computer Networking and Data Communication
Networking Utility Programs Reference

ip/ifconfig – show/manipulate routing, network devices, interfaces and tunnels

Overview

The ip command (man page) is the Swiss-army knife of Linux networking commands. It can both show and configure your network interfaces, MAC and IP addresses, routes, and more.[1] It has the following command format:

```
› ip [ OPTIONS ]  OBJECT  { COMMAND | help }
```

Example

To start, view the available network devices on your machine using the following command:

```
› ip -brief link show
```

```
~ $ ip -brief link show
lo               UNKNOWN        00:00:00:00:00:00 <LOOPBACK,UP,LOWER_UP>
enp3s0f0         UP             e4:3d:1a:a0:2d:6a
<BROADCAST,MULTICAST,UP,LOWER_UP>
enp3s0f1         UP             e4:3d:1a:a0:2d:6b
<BROADCAST,MULTICAST,UP,LOWER_UP>
enp0s31f6        DOWN           74:86:e2:28:9d:ec <NO-
CARRIER,BROADCAST,MULTICAST,UP>
virbr0           DOWN           52:54:00:25:3a:b5 <NO-
CARRIER,BROADCAST,MULTICAST,UP>
```

We use the `-brief` option to hide extra details that we don't need. The `link` object relates to the actual network devices available on your machine (such as both wired and wireless network interface cards). Finally, the `show` command prints out relevant information for the object we put in our command. The result will include a row for each device:

- The first column indicates the name given to the network device.
- The second column indicates the status (e.g. DOWN or UP).
- The third column shows the MAC address for the device.
- The final column shows the interface's flags and features.

Next, we can try out the `address` object for the `ip` command. This will tell us information such as the IP address assigned to that device.

```
› ip -brief addr show
```

```
lo               UNKNOWN        127.0.0.1/8
enp3s0f0         UP             192.168.254.7/24
enp3s0f1         UP             fe80::e63d:1aff:fea0:2d6b/64
enp0s31f6        DOWN
virbr0           DOWN           192.168.122.1/24
```

[1] **Changes made by the `ip` command do not persist on reboot.** This is useful for labs (we don't want to make permanent changes, and we can always recover from mistakes). Nonetheless, it is still important to remember this when configuring any Linux machines in the future. To make permanent changes, consult the documentation for your specific Linux distribution. Most modern Linux distros now use the NetworkManager daemon for network configuration.

ping – send ICMP packets to network hosts

Overview

The ping command (man page) is a *very* useful, simple command for determining if two hosts (i.e. computers) on a network can communicate with one-another. It generally has the following format:

```
› ping [ OPTIONS ]  DESTINATION
```

Example

To start, let's ping www.iastate.edu. We'll use the `-i 3` option to specify that we want to send (and receive) 3 ping packets, then quit. You can alternatively omit this and use CTRL + C to end it manually.

```
› ping -c 3 www.iastate.edu
```

```
PING www.iastate.edu (20.241.39.52): 56 data bytes
64 bytes from 20.241.39.52: icmp_seq=0 ttl=114 time=20.475 ms
64 bytes from 20.241.39.52: icmp_seq=1 ttl=114 time=13.902 ms
64 bytes from 20.241.39.52: icmp_seq=2 ttl=114 time=20.729 ms

--- www.iastate.edu ping statistics ---
3 packets transmitted, 3 packets received, 0.0% packet loss
round-trip min/avg/max/stddev = 13.902/18.369/20.729/3.160 ms
```

Pay most attention to the *time* at the end of each packet, and at the bottom. Ping has two main purposes: check for connectivity, and check for the *latency* of that connectivity. In other words, how long does it take to send and receive a single packet to/from my destination? *Jitter* is another term commonly used to describe the variance in latency. If your latency frequently jumps up and down, you could infer that there is a lot of jitter between you and the destination.

traceroute – print the route packets take to network host

Overview

The traceroute command is a clever utility that figures out the route packets will likely take between your machine and another machine. Each step along the journey is considered a “hop”. The first hop that isn’t your machine is considered the “gateway” (in your home network, this is your router).

Traceroute functions by sending a series of packets with increasing time-to-live (TTL) values. To start, it’ll send a packet with a TTL of 1 to your destination. After this packet reaches the gateway, the gateway will notice the TTL has expired, drop the packet, and notify your computer with a “Time Exceeded” packet (a courtesy, thanks gateway!). It just so happens that these responses include the hop’s IP address. This clever way of prompting a response is how traceroute determines the address and latency to each hop. To find the IP of the second hop, traceroute sends another handful of packets with a TTL of 2. Then, TTL=3 and so on…

To run traceroute, we can use the following command syntax:

```
› traceroute [ OPTIONS ]  DESTINATION
```

Example

To start, let’s run a traceroute to ol’ reliable, 1.1.1.1 (Cloudflare’s DNS).

```
› traceroute 1.1.1.1
```

```
traceroute to 1.1.1.1 (1.1.1.1), 30 hops max, 60 byte packets
1  _gateway (192.168.254.254)  0.550 ms  0.526 ms  0.523 ms
2  routera-129-186-5-0.tele.iastate.edu (129.186.5.252)  4.522 ms  4.572 ms
4.680 ms
3  b31-mpls-p-hu0-3-0-9--to--b11-mpls-pe-eth1-1.tele.iastate.edu
(129.186.0.186)  1.063 ms  1.093 ms  1.095 ms
4  b31-mpls-fpe-eth2-10--to--e63-mpls-p-hu0-2-0-1.tele.iastate.edu
(129.186.0.141)  1.268 ms e63-mpls-fpe-eth1-10--to--b31-mpls-p-hu0-3-0-
1.tele.iastate.edu (129.186.0.137)  1.161 ms b31-mpls-fpe-eth2-10--to--e63-
mpls-p-hu0-2-0-1.tele.iastate.edu (129.186.0.141)  1.470 ms
5  e63fr--b31fpe-vrf-data.tele.iastate.edu (129.186.254.245)  1.280 ms b31fr-
-e63fpe-vrf-data.tele.iastate.edu (129.186.254.247)  1.080 ms e63fr--e63fpe-
vrf-data.tele.iastate.edu (129.186.254.253)  1.046 ms
6  b31be-eth1-2.fusion.tele.iastate.edu (192.188.159.227)  1.245 ms  1.206 ms
e63be-eth1-2.fusion.tele.iastate.edu (192.188.159.229)  1.168 ms
7  routerb-192-188-159-96.tele.iastate.edu (192.188.159.101)  1.016 ms  0.970
ms  0.998 ms
8  rtr-e63be-vlan933.tele.iastate.edu (192.188.159.106)  1.570 ms  1.598 ms
1.767 ms
9  rtr-e63isp1-be154.tele.iastate.edu (192.188.159.155)  2.619 ms  3.012 ms
3.672 ms
10  rtr-b31isp1-be162.tele.iastate.edu (192.188.159.162)  2.104 ms  2.300 ms
1.263 ms
11  164.113.254.205 (164.113.254.205)  6.008 ms  6.455 ms  6.453 ms
12  cloudflare.gnd1.mci.kcix.net (206.51.7.34)  6.515 ms  6.494 ms  6.481 ms
13  172.68.148.19 (172.68.148.19)  6.736 ms 172.68.148.6 (172.68.148.6)
6.379 ms  6.474 ms
14  one.one.one.one (1.1.1.1)  6.143 ms  6.081 ms  6.175 ms
```

Traceroute will report the hop count, the hostname (if available), and three different *cumulative* round-trip time measurements for each hop (e.g., it took about 6ms for our packets to reach 1.1.1.1 from our machine) . It's possible that each of your packets at a given hop get routed through a different router—if so, you'll see multiple hostnames or addresses for a single hop. Notice that our first 10 hops are just within Iowa State's network!

Often, you'll see asterisks reported by traceroute. This simply means we did not get a response from that hop. That does *not necessarily* mean that that hop *isn't* routing our packets. It's very possible that the router has been configured to not respond with TTL-expired packets but still route packets with sufficient TTL. Sometimes, you can get around this by directing traceroute to use TCP SYN packets instead of ICMP packets. This can be done by either supplying the `--tcp` option to traceroute or by using `tcptraceroute` instead (you'll need to run this as root with the 489labuser account):

```
› sudo tcptraceroute google.com
```

nslookup – query Internet name servers (DNS information)

Overview

The nslookup command (man page) can be used to retrieve IP addresses and other information (such as 20.241.39.52) for given domain names (such as www.iastate.edu).

```
› nslookup [ OPTIONS ]  [ NAME ]  [ SERVER ]
```

We'll learn about the DNS protocol and name resolution later in the course. For now, know that special internet servers exist which will resolve domain names to IPs for your computer. These "nameservers" contain tons of DNS records of different types. A table of types supported by nslookup is included below. You may specify the query option to query different records for a name, e.g. `-query=MX` for mail exchanger records.

A	the host's Internet address
CNAME	the canonical name for an alias
HINFO	the host CPU and operating system type
MINFO	the mailbox or mail list information
MX	the mail exchanger
NS	the name server for the named zone
PTR	the host name if the query is an Internet address; otherwise, a pointer to other information
SOA	the domain's "start-of-authority" information
TXT	the text information
WKS	the supported well-known services

Common and popular DNS servers include 1.1.1.1 (operated by CloudFlare) and 8.8.8.8 (operated by Google). However, Internet Service Providers (ISPs) as well as large businesses and enterprises will often operate their own DNS servers for security reasons. DNS is a collaborative effort between many different DNS servers operating at different levels of the DNS hierarchy.

Your computer is frequently performing DNS lookups behind the scenes! For example, when you type in www.google.com into your web browser, your computer contacts a known nameserver and asks it, "What is the IP address for the website www.google.com?" Think of the nameserver like a detective who will hunt down this information for you. Eventually, it'll tell you something like "www.google.com is really just 142.250.191.132." Your computer can then send its requests and traffic to 142.250.191.132, one of Google's *many* servers.

Example

To start, let's do a manual lookup for www.microsoft.com.

```
› nslookup www.google.com
```

```
Server: 8.8.8.8
Address:        8.8.8.8#53

Non-authoritative answer:
www.microsoft.com       canonical name = www.microsoft.com-c-3.edgekey.net.
www.microsoft.com-c-3.edgekey.net       canonical name = www.microsoft.com-c-
3.edgekey.net.globalredir.akadns.net.
www.microsoft.com-c-3.edgekey.net.globalredir.akadns.net        canonical name =
e13678.dscb.akamaiedge.net.
Name:   e13678.dscb.akamaiedge.net
```

Address: 23.205.97.201

Microsoft's website is a bit more interesting than some others! At the top, we see the nameserver used (in this case, nslookup used Google's DNS, 8.8.8.8).

"Non-authoritative" just means that the nameserver used for our lookup isn't one of the authoritative servers that www.microsoft.com is registered on (which is expected and OK).

Next, we see a whole list of "canonical names". This tells us that www.microsoft.com is just an alias (another name) for www.microsoft.com-c-3.edgekey.net. That, in turn, is an alias for www.microsoft.com-c-3.edgekey.net.globalredir.akadns.net, and so on and so forth. Eventually, we find out that www.microsoft.com is just an alias for and has the canonical name e13678.dscb.akamaiedge.net with the IP address 23.205.97.201.

In practice, we can infer that Microsoft uses another company named Akamai for hosting and load-balancing their site. If Microsoft wanted to switch providers, they could change the DNS CNAME (canonical name) type records for www.microsoft.com to instead point to other domain names. If they wanted to host it themselves, they could *not* have CNAME records and simply have an A (address) record which maps www.microsoft.com to an IP address.

iPerf – perform network traffic tests

Overview

iPerf (website) is a great program for measuring throughput among other metrics between two machines. In order to run iperf, one machine must act as a "server" and run the server command.

```
› iperf -s
```

The other machine must run the client command and specify the IP address of the server.

```
› iperf -c server.ip.address.here
```

The test will begin. After each interval, both the server and client will print statistics for that interval.[2]

Example

Let's run an iPerf test from the machine with an address 192.168.254.7 against the machine with an address 192.168.254.8:

```
› iperf -s                      (on the machine with 192.168.254.8)
› iperf -c 192.168.254.8  (on the machine with 192.168.254.7)
```

```
------------------------------------------------------------
Client connecting to 192.168.254.8, TCP port 5001
TCP window size: 85.0 KByte (default)
------------------------------------------------------------
[  1] local 192.168.254.7 port 53984 connected with 192.168.254.8 port 5001
[ ID] Interval       Transfer     Bandwidth
[  1] 0.00-10.03 sec  1.10 GBytes   939 Mbits/sec
```

We can see that one 10.03 second interval was performed, and 1.10 GBytes was transferred during that time. iPerf reports the bandwidth as 939 Mbits/sec, which would be healthy for a connection on a 1-Gigabit network.

[2] The interval can be specified with the -i option. The total time of the test can be specified with the -t option. The default is usually a single 10-second interval.

Nmap – scan networks to discover hosts and open ports

Overview

Nmap (Network Mapper) is a security scanner originally written by Gordon Lyon (also known by his pseudonym Fyodor Vaskovich) used to discover hosts and services on a computer network, thus creating a "map" of the network. To accomplish its goal, `nmap` sends specially crafted packets to the target host and then analyzes the responses. Many systems and network administrators also find it useful for tasks such as network inventory, managing service upgrade schedules, and monitoring host or service uptime.

Example

Let's map out open ports on a specific host, 129.186.215.41. The -Pn option, as you can read in the man page for Nmap, simply instructs Nmap to skip its traditional check called "host discovery". In layman's terms, we tell Nmap it doesn't need to check if 129.186.215.41 is online first.

```
› nmap -Pn 129.186.215.41
```

```
Starting Nmap 6.40 ( http://nmap.org ) at 2017-08-22 15:46 CDT
Nmap scan report for bones.ee.iastate.edu (129.186.215.41)
Host is up (0.00054s latency).
Not shown: 989 closed ports
PORT    STATE SERVICE
21/tcp  open  ftp
22/tcp  open  ssh
23/tcp  open  telnet
25/tcp  open  smtp
79/tcp  open  finger
110/tcp open  pop3
111/tcp open  rpcbind
143/tcp open  imap
587/tcp open  submission
700/tcp open epp

Nmap done: 1 IP address (1 host up) scanned in 5.85 seconds
```

The output shows us that a handful of TCP ports are open on the host, as well as the services running on those ports.

WARNING: Nmap is a helpful security tool, but it's also often the first step used in attacks. Perform port scans **only on machines that you have been given permission to do so**.

tcpdump – monitor incoming and outgoing traffic

Overview

`tcpdump` is a command line tool for analyzing raw network traffic; every packet going through the network interface card is captured (i.e., `tcpdump` is a packet sniffer). This tool is commonly used by developers to debug network applications and by network administrators to log network traffic for later analysis. All network traffic received by the network interface is captured by `tcpdump`, including traffic that is not related to the system running `tcpdump`.

Example

`tcpdump` must be started from the command line and requires root privileges to run. To use it, you must become the root user (using `sudo`). Start `tcpdump` on the enp3s0f0 interface:

```
› sudo /usr/sbin/tcpdump -i enp3s0f0
```

`tcpdump` will start dumping the headers of all packets received by the network interface `enp3s0f0` to the terminal. Depending on the amount of traffic, the information given by `tcpdump` can quickly become overwhelming. Press `Control + C` to stop `tcpdump`.

Filtering Traffic

In reality, we might only be interested in checking specific network traffic, not all of it. This is why we use the different filters available in `tcpdump`. (When running these commands with Coover 2061 computers, be sure to use the full path of `tcpdump` with `sudo`, i.e., `sudo /usr/sbin/tcpdump`.) Common filters include:

Host

The host filter will filter out all traffic sent or received to a certain host. The two examples below show how to log traffic to host 129.186.1.200 and to a known machine name.

```
› tcpdump host 129.186.1.200
› tcpdump host ns-1.iastate.edu
```

Source and Destination Address

These work the same as host, except you can explicitly filter either source or destination traffic. For example, to log all traffic sent to your host, you can use

```
› tcpdump dst <IP address>
```

Subnet

This will capture an entire network segment's traffic. For example, if you wanted to capture all traffic on subnet 129.186.1.0/24 (i.e. the network between 129.168.1.0 and 129.186.1.255).

```
› tcpdump net 129.186.1.0/24
```

Protocol

Filter based on the network protocol. Supported protocols include tcp, udp, icmp, arp, rarp and other. For example, to view all tcp traffic on the network:

```
› tcpdump tcp
```

Note: You do not type "proto" before the protocol type.

Port

Filter based on the TCP or UDP port. For example, to view all port 80 (HTTP) traffic:

```
› tcpdump port 80
```

We can also filter by the port used by just the source or destination machine using `src` or `dst`. For example, to view all incoming port 21 (FTP) traffic:

```
› tcpdump dst port 21
```

Combination Filters

The power of filters can be enhanced even further by combining them. By combining multiple filters, a very specific subset of network traffic can be logged. Filters are combined using the logical operators *and*, *or*, and *not*. Some examples are as follows:

View all tcp traffic from the machine 192.168.0.2 destined for port 21:

```
› tcpdump tcp and src 192.168.0.2 and dst port 21
```

Examine all traffic originating from the 129.186.158.0 network destined for the 192.168.1 network:

```
› tcpdump src net 129.186.158.0/23 and dst net 192.168.1.0/24
```

Examine all traffic from your host that is not the ICMP protocol:

```
› tcpdump src <IP address> and not icmp
```

Show only ICMP traffic that is not an echo request (8) or an echo reply (0):

```
› tcpdump 'icmp[0]!= 8 and icmp[0] !=0'
```

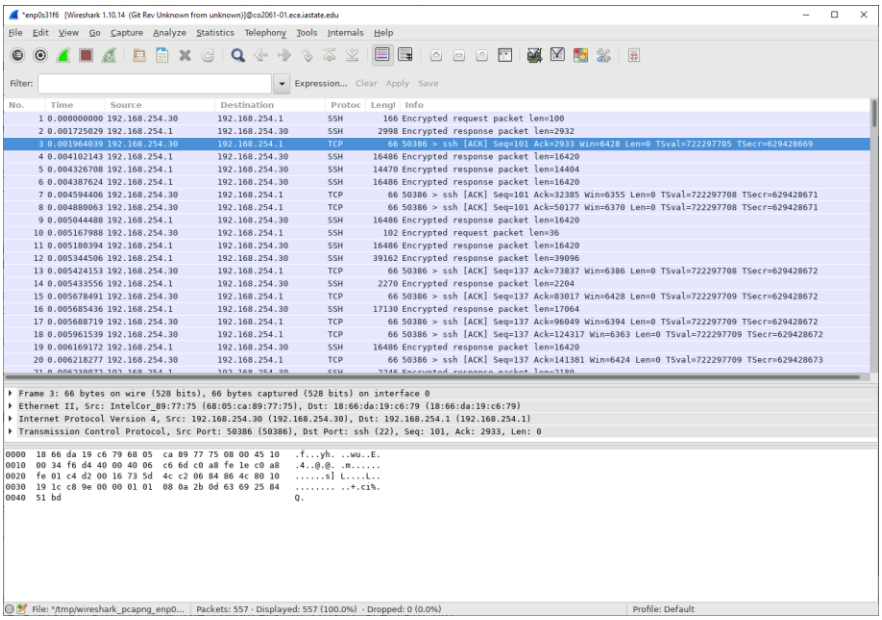
Wireshark – GUI Program for monitoring network traffic

Overview

Wireshark, the successor to Ethereal, is an open-source network analyzer. Like `tcpdump`, it's considered a packet sniffer. Wireshark is functionally similar to `tcpdump`; however, it has a graphical user interface and many more filtering options than `tcpdump`. Wireshark is heavily used when trying to debug the network from a host perspective, and its powerful feature set makes it popular throughout industry.

Usage

Wireshark can be started by typing `wireshark` at the command line. After typing the command, you should get the GUI version of Wireshark.



First, notice that Wireshark has three panes:

- The top pane is the packet list pane. It displays a summary of each captured packet. Click a packet in this pane to display more detailed information about it in the other two panes.
- The middle pane is the tree view pane. It displays more information about the packet selected in the packet list pane. It's organized hierarchically, with the lower network layers at the top of the pane.
- The bottom pane is data view pane. It displays the packet selected in the packet list pane as hex and ASCII. Also, it highlights the data associated with the field selected in the tree view pane.

To use Wireshark to sniff the wire, you first need to select the device used to communicate with the network (most often eth0, but in our case we'll be using `enp3s0f0`). After you do this, Wireshark will capture all packets that go to and out of the interface card. To accomplish this, follow these steps:

- Go to **Capture → Options**. Make sure `enp3s0f0` is selected as the interface at the top of the Options window.
- Now, make sure the "**Use promiscuous mode on all interfaces**" box is checked. This will allow the network interface card to capture any packets on the wire (as opposed to just the packets destined for your machine).
- Click **Start**. After about 10 seconds, click **Stop**. Take a look at what was captured: source, protocol, time, etc. Since the results are usually plenty, you can also filter through them using

preset filters, or even your own custom filters. Click on the **Filter** button and select "**TCP only**" from the list. Now look at the capture list and see how your results are filtered.